⚠ CONFIDENTIAL — INTERNAL USE ONLY

TUM2026 Security Audit Report

Full Application Security Assessment · Second Review

REPORT DATE	APPLICATION	AUDITOR
March 13, 2026	TUM2026 IT Admin Dashboard	Claude Code (Automated)
STACK	**DEPLOYMENT**	**PRIOR AUDIT**
Next.js 16 + Prisma 7 + NextAuth v5	Vercel + Neon.tech	March 12, 2026

6	7	6
CRITICAL FINDINGS	HIGH FINDINGS	MEDIUM FINDINGS

1 Executive Summary

This report presents findings from a full security audit of the TUM2026 multi-tenant IT Admin Dashboard, conducted on March 13, 2026. The audit examined authentication logic, access control, input validation, production hygiene, rate limiting, and security headers across the entire API surface (~70 route handlers and 15+ server actions).

The application is built on Next.js 16 App Router, Prisma 7, NextAuth v5 (Microsoft Entra ID SSO), and deployed on Vercel with a Neon.tech PostgreSQL database. It manages IT operations across multiple tenants including Active Directory operations (via SentinelAgent), NinjaOne RMM, and Sophos Central.

⚠ **Critical Risk:** The most severe finding is a pervasive IDOR (Insecure Direct Object Reference) pattern affecting at least 19 API routes. Any authenticated user — regardless of which tenants they are permitted to access — can read and modify data from *any* tenant by supplying a different tenant ID in the request. This includes the ability to disable AD accounts, reset passwords, reboot devices, and delete device links across all managed organizations.

6	7	6	2
CRITICAL	HIGH	MEDIUM	LOW / PASS

SOC-2

~35%

CC6.6 violated by IDOR pattern

HIPAA

~50%

Missing BAAs, no session expiry

CMMC Level 2

~30%

AC.1.002 violated by IDOR

2 Critical Findings

C-01 IDOR — Tenant-scoped AD routes lack tenant ownership verification

CRITICAL

AFFECTED FILES

`src/app/api/tenants/[id]/ad/users/route.ts` `src/app/api/tenants/[id]/ad/groups/route.ts`
`src/app/api/tenants/[id]/ad/users/[sam]/route.ts`
`src/app/api/tenants/[id]/ad/groups/[name]/members/route.ts`
`src/app/api/tenants/[id]/ad/sync/route.ts` `src/app/api/tenants/[id]/licenses/route.ts`
`src/app/api/tenants/[id]/device-assignments/route.ts` `src/app/api/tenants/[id]/users/route.ts`

WHY IT MATTERS

All routes authenticate the user (`session?.user`) but never verify the user has permission for that *specific* tenant. Any authenticated technician can enumerate users from all tenants, view AD groups and memberships cross-tenant, read license assignments, and device assignments — simply by changing the `[id]` path parameter. The POST endpoint at `/api/tenants/[id]/ad/users/[sam]` additionally allows cross-tenant disable, enable, unlock, and password reset of AD accounts — a direct operational impact to managed clients.

VULNERABLE CODE PATTERN (CONFIRMED IN SOURCE)

```
export async function GET(req, { params }) {
  const session = await auth();
  if (!session?.user) {
    return NextResponse.json({ error: "Unauthorized", status: 401 });
  }
  const { id } = await params;
  const tenantId = Number(id);
  // ← NO tenant ownership check here
  const users = await prisma.adUser.findMany({ where: { tenantId } });
  return NextResponse.json(users); // Returns data for ANY tenant
}
```

FIX

Create `src/lib/tenant-auth.ts` with a `requireTenantAccess(session, tenantId)` helper that checks `UserPermission` table for non-ADMIN users. Apply as the first call in every tenant-scoped route handler.

C-02 IDOR — NinjaOne device operations not scoped to any tenant

CRITICAL

AFFECTED FILES

`src/app/api/ninja/devices/[deviceId]/reboot/route.ts`

`src/app/api/ninja/devices/[deviceId]/patch-apply/route.ts`

`src/app/api/ninja/devices/[deviceId]/patch-scan/route.ts`

`src/app/api/ninja/devices/[deviceId]/script/route.ts`

`src/app/api/ninja/devices/[deviceId]/services/[serviceId]/control/route.ts`

`src/app/api/ninja/devices/[deviceId]/maintenance/route.ts`

WHY IT MATTERS

These endpoints accept a raw NinjaOne `deviceId` (integer) and immediately perform the operation without verifying the device belongs to a tenant the user has access to. An EDITOR user can reboot, patch, run scripts, or control services on *any device in any NinjaOne organization* managed by the platform. This includes production servers and workstations of all clients.

VULNERABLE CODE PATTERN (CONFIRMED IN SOURCE)

```
// reboot/route.ts — same pattern in all 6 files
const role = (session.user as Record<string, unknown>).role as string;
if (role !== "ADMIN" && role !== "EDITOR") { ... }

const deviceId = Number(raw);
// ← No DB lookup to verify device → tenant → user permission
await rebootDevice(deviceId, mode, reason); // Works for any deviceId
```

FIX

Add a `DeviceAssignment` lookup before the operation: verify the device's `tenantId` from the DB, then call `requireTenantAccess(session, tenantId)`. Alternatively, require `?tenantId=` as a query param and validate it.

C-03 IDOR — Cross-link deletion has no ownership check

CRITICAL

FILE

`src/app/api/cross-links/[id]/route.ts`

WHY IT MATTERS

Any authenticated user can delete any `DeviceCrossLink` record by guessing or iterating its integer auto-increment ID. Cross-links represent the NinjaOne↔Sophos bidirectional device mapping; destroying them disrupts cross-platform monitoring for all clients. The delete is permanent — no soft-delete or audit recovery exists.

CODE

```
await prisma.deviceCrossLink.delete({ where: { id } });
// No WHERE tenantId check; no role check; deletes any record
```

FIX

First fetch the record to get its `tenantId`, then call `requireTenantAccess(session, tenantId)`. Also require minimum EDITOR role.

C-04 IDOR — Auto-match accepts arbitrary tenantId without ownership check

CRITICAL

FILE

`src/app/api/cross-links/auto-match/route.ts`

WHY IT MATTERS

The body-provided `tenantId` is used directly to fetch all NinjaOne devices and Sophos endpoints for that tenant. This exposes the full device inventory and endpoint list of any tenant to any authenticated user. Additionally, the paginated Sophos fetch has no upper page limit, allowing a user to trigger hundreds of external API calls that count against rate limits and could exhaust third-party API quotas.

FIX

Call `requireTenantAccess(session, tenantId)` after parsing the request body. Add a `maxPages = 20` guard in the Sophos pagination loop.

C-05 Global user search returns data from ALL tenants regardless of user permissions

CRITICAL

FILE

`src/app/api/search/route.ts`

WHY IT MATTERS

The search endpoint fetches `prisma.tenant.findMany()` with no filter, then calls `getUsers(tenant.id)` for every tenant and returns up to 20 results aggregated across all of them. A VIEWER assigned only to Tenant A can search for employees at Tenant B, C, D, etc. — exposing names, emails, and organizational structure that may be confidential (e.g., mergers, acquisition targets, healthcare staff).

CODE

```
const tenants = await prisma.tenant.findMany({ ... }); // ALL tenants
const results = await Promise.allSettled(
  tenants.map(async (tenant) => {
    const users = await getUsers(tenant.id); // Fetches from each tenant
    return users.filter(u => matches(u, q)).map(u => ({ ...u, tenantId: tenant.id }));
  })
);
```

FIX

Filter the initial `tenant.findMany()` to only include tenants the current user has access to (via `UserPermission` join). ADMINs may bypass this filter.

C-06 Audit log export accessible by any authenticated user — no role check

CRITICAL

FILE

`src/app/api/logs/export/route.ts`

WHY IT MATTERS

The CSV export downloads up to 10,000 audit log entries and requires only a valid session cookie — no role check whatsoever. Audit logs contain actor emails, entity IDs, timestamps, and operation details for all tenants. A low-privilege VIEWER can download the full audit history, which reveals every administrative action ever taken on the platform — useful for attackers planning targeted privilege escalation or covering tracks.

CODE

```
export async function GET(request: NextRequest) {
  const session = await auth();
  if (!session) return new Response("Unauthorized", { status: 401 });
  // ← No role check. Any session = full audit log access
  const logs = await prisma.auditLog.findMany({ take: 10000 });
  return new Response(csv, { headers: { 'Content-Disposition': 'attachment;...' } });
}
```

FIX

Add `requireRole("ADMIN")` before querying. Audit exports are ADMIN-only operations per SOC-2 CC4.1.

3 High Findings

H-01 Missing security headers — no CSP, HSTS, X-Frame-Options

HIGH

FILE

`next.config.ts`

WHY IT MATTERS

The `next.config.ts` defines no HTTP response headers. Without these headers: (1) **Clickjacking** — the app can be embedded in an iframe on a malicious site; (2) **MIME sniffing** — browsers may misinterpret response types, enabling script injection; (3) **XSS** — without CSP, injected scripts execute freely; (4) **Protocol downgrade** — without HSTS, users can be MitM'd on first visit. Production is on Vercel HTTPS but headers still missing at app layer.

FIX

Add a `headers()` function to `next.config.ts` with: `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy: geolocation=()`, and a strict CSP. Vercel also supports headers in `vercel.json`.

H-02 No rate limiting on any API endpoint

HIGH

SCOPE

All ~70 API routes — confirmed via `package.json` (no rate-limiting package installed)

WHY IT MATTERS

No rate limiting means: (1) **Enumeration attacks** — iterate tenant IDs 1–999 to discover all tenant data; (2) **Credential stuffing** — brute-force session tokens; (3) **API quota exhaustion** — trigger bulk NinjaOne/Sophos API calls that burn third-party rate limits; (4) **DoS** — flood the serverless functions causing cold-start cascades and billing spikes. In-memory rate limiters won't work on Vercel (stateless serverless); must use a distributed solution.

FIX

Use `@upstash/ratelimit` with Upstash Redis (has a free tier). Apply at middleware level for all `/api/*` routes: 60 req/min per IP for reads, 10 req/min for write/action endpoints (reboot, patch-apply, AD mutations).

H-03 Inconsistent RBAC — NinjaOne device actions use string comparison instead of RBAC helper

HIGH

AFFECTED FILES

`src/app/api/ninja/devices/[deviceId]/reboot/route.ts`

`src/app/api/ninja/devices/[deviceId]/patch-apply/route.ts`

`src/app/api/ninja/devices/[deviceId]/patch-scan/route.ts`

`src/app/api/ninja/devices/[deviceId]/script/route.ts`

WHY IT MATTERS

These routes use `role !== "ADMIN" && role !== "EDITOR"` with an unsafe `as string` cast, while Sophos routes correctly use `getSessionRole()` and `hasMinRole()` from `rbac.ts`. If a future Role type is added (e.g., `TECH`), the hardcoded checks will silently block valid users or — if the Role enum is modified — fail open.

FIX

Replace all manual role checks with `const role = await getSessionRole(); if (!hasMinRole(role, "EDITOR")) { ... }` to match the Sophos pattern.

H-04 Middleware validates session cookie existence but not session validity or role

HIGH

FILE

`src/middleware.ts`

WHY IT MATTERS

The middleware only checks for the *existence* of the cookie name (`authjs.session-token`) without cryptographically validating it. A user with a forged or expired cookie name (but no valid signature) passes middleware and reaches route handlers. Route handlers do call `await auth()` themselves, so this is defense-in-depth issue — but it means the middleware provides no actual security guarantee, only a UX redirect. Additionally, role-based page restrictions (e.g., `/dashboard/permissions` should be ADMIN-only) are enforced only client-side, not in middleware.

FIX

Use the NextAuth `auth` export directly in middleware (it supports Edge runtime in Auth.js v5). This validates the JWT signature. Add page-level role checks in middleware for known admin-only paths.

H-05 Server actions lack input validation — no Zod schemas despite Zod being installed

HIGH

AFFECTED FILES

`src/app/dashboard/tenants/new/actions.ts` `src/app/dashboard/permissions/actions.ts` `src/app/dashboard/tasks/actions.ts` `src/app/dashboard/technicians/actions.ts`

WHY IT MATTERS

Server actions accept raw `FormData` and apply only basic `.trim()` checks before passing to Prisma. No length limits, no character set restrictions, no UUID format validation for `tenantIdMsft` / `tenantIdRewst`. A malicious actor could insert very long strings, Unicode control characters, or invalid UUIDs that cause downstream Graph API or database errors. `zod@4.3.6` is already installed and unused.

FIX

Create `src/lib/schemas/tenant.ts`, `permission.ts`, etc. with Zod schemas. Apply `schema.safeParse(Object.fromEntries(formData))` at the start of each action. Phase 4 of the SOC-2 plan already specifies this work.

H-06 Security and health reports return all-tenant data regardless of user permissions

HIGH

AFFECTED FILES

`src/app/api/reports/security/route.ts` `src/app/api/reports/health/route.ts` `src/app/api/reports/licenses/route.ts`

WHY IT MATTERS

These endpoints fetch `prisma.tenant.findMany()` without filtering by user permissions, then aggregate security scores, health metrics, and license data across all tenants. A VIEWER technician authorized only for one tenant can access sensitive security posture and license cost data for all other organizations.

FIX

Filter the tenant list to only those the user has access to via `UserPermission`. ADMIN role bypasses filter (can see all). This is the same fix needed for the search endpoint (C-05).

H-07 Audit activity analytics accessible to any authenticated user

HIGH

FILE

`src/app/api/analytics/audit-activity/route.ts`

WHY IT MATTERS

Returns timestamp-grouped audit log counts for a configurable window (up to any number of days via `?days=` query param). Only checks `if (!session)`. No upper limit on the `days` parameter — a user could request the entire audit history by specifying `?days=99999`, generating a large aggregation query.

FIX

Require minimum EDITOR role. Cap `days` to a maximum of 90 or 365.

4 Medium & Low Findings

M-01 AD mutation routes missing RBAC role check

MEDIUM

FILE

`src/app/api/tenants/[id]/ad/users/[sam]/route.ts` — POST method

WHY IT MATTERS

The POST handler (disable/enable/unlock/reset-password) only checks `session?.user` — it does not require EDITOR or ADMIN role. A VIEWER role technician can perform AD user mutations if they know the tenant ID and SAM account name.

FIX

Add `if (!hasMinRole(role, "EDITOR")) return 403` before the action handler in the POST method.

M-02 Search query has no maximum length limit

MEDIUM

FILE

`src/app/api/search/route.ts:11`

WHY IT MATTERS

The minimum length check `q.length < 2` guards against empty queries but there is no upper bound. A query of arbitrary length is lowercased and sent to all tenant Graph API calls in parallel. While Graph API has its own limits, the overhead of processing a large string through 15+ parallel API calls is unnecessary.

FIX

Add `|| q.length > 100` to the early-return check.

M-03 Custom field values accept entity IDs without ownership validation

MEDIUM

FILE

`src/app/api/custom-fields/values/route.ts`

WHY IT MATTERS

The PUT endpoint accepts `{ fieldId, entityType, entityId, value }` from the request body. When `entityType="tenant"`, the `entityId` is a tenant ID. No check verifies the caller has permission for that tenant. Users can write custom field values to any tenant's record.

FIX

When `entityType === "tenant"`, call `requireTenantAccess(session, Number(entityId))` before the upsert.

M-04 Auto-match Sophos pagination has no max-pages guard

MEDIUM

FILE

`src/app/api/cross-links/auto-match/route.ts:63-72`

WHY IT MATTERS

The `while (hasMore)` loop fetches 500 endpoints per page with no page ceiling. For a large organization with thousands of endpoints, this could run for minutes, exhaust Sophos API rate limits, and cause Vercel function timeouts (max 300s on Pro plan).

FIX

Add `const MAX_PAGES = 20; let pagesRead = 0;` and break when `pagesRead ≥ MAX_PAGES`. Return a `truncated: true` flag in the response if the limit was hit.

M-05 Sophos scan/tamper routes accept arbitrary tenantId from query string without ownership check

MEDIUM

FILES

`src/app/api/sophos/endpoints/[endpointId]/scan/route.ts`

`src/app/api/sophos/endpoints/[endpointId]/tamper/route.ts`

WHY IT MATTERS

These routes correctly use `hasMinRole(role, "EDITOR")` but accept `?tenantId=` from the query string without verifying the user has access to that tenant. An EDITOR can trigger Sophos scans or toggle tamper protection on endpoints in any tenant.

FIX

Add `requireTenantAccess(session, tenantId)` after parsing `tenantIdRaw`.

M-06 SecretsManager (Bitwarden) not yet integrated into auth.ts — credentials still read from env vars

MEDIUM

FILE

`src/lib/auth.ts`

WHY IT MATTERS

The Bitwarden SecretsManager implementation in `src/lib/secrets.ts` is complete (per previous audit), but `auth.ts` still reads `AUTH_MICROSOFT_ENTRA_ID_ID` and `AUTH_MICROSOFT_ENTRA_ID_SECRET` directly from `process.env`. This is Phase 1 of the SOC-2 plan and remains unfinished. The secrets are also stored in `.env.local` and were previously exposed in git history (note: git-filter-repo purge was applied in March 2026, but rotation is still pending).

FIX

Complete Phase 1 of SOC-2 plan: implement async NextAuth initialization using `src/lib/auth-init.ts`. Rotate `AUTH_MICROSOFT_ENTRA_ID_SECRET` and `GRAPH_CLIENT_SECRET` in Azure Portal immediately.

Low / Informational

L-01 NinjaOne base URL and scope have hardcoded defaults in source code

LOW

FILE

`src/lib/ninja.ts`

NOTE

Non-sensitive defaults. Move to `.env.local.example` for clarity. Not exploitable.

PASS No console.log with sensitive data in production code

PASS

NOTE

The 9 console.log/warn/error statements identified in the March 12 audit were successfully removed. No further occurrences found in source files.

PASS No secrets in NEXT_PUBLIC_ env vars or client-side code

PASS

NOTE

No `NEXT_PUBLIC_` variables found containing tokens or secrets. The Bitwarden SecretsManager is server-only and not imported in any `"use client"` component.

5 Remediation Roadmap

Findings are grouped by remediation effort. Items in Phase A share a common fix (`requireTenantAccess` helper) and should be implemented together to maximize efficiency.

Phase A — Immediate (≤ 1 day): Create `requireTenantAccess` helper and apply to all IDOR routes

Finding	File	Action
C-01	`src/lib/tenant-auth.ts` (new)	**Create `requireTenantAccess(session, tenantId)`**
C-01	All `tenants/[id]/**` routes	**Apply `requireTenantAccess` after tenantId parse**
C-02	`ninja/devices/[deviceId]/*`	**Lookup device → tenant, apply `requireTenantAccess`**
C-03	`cross-links/[id]/route.ts`	**Fetch record first, then `requireTenantAccess`**
C-04	`cross-links/auto-match/route.ts`	**Apply `requireTenantAccess` + max-pages guard**
C-05	`search/route.ts`	**Filter tenants by UserPermission**
M-03	`custom-fields/values/route.ts`	Add entity ownership check for entityType="tenant"
M-05	`sophos/endpoints/*/scan + tamper`	Apply `requireTenantAccess` to tenantId query param

Phase B — Same Day: Access control hygiene

Finding	File	Action
C-06	`logs/export/route.ts`	**Add `requireRole("ADMIN")`**
H-06	`reports/security + health + licenses`	**Filter tenants by UserPermission for non-ADMINs**
H-07	`analytics/audit-activity/route.ts`	**Add EDITOR min role; cap `days` to 365**
H-03	`ninja/devices/[deviceId]/*`	**Replace string role checks with `hasMinRole()`**
M-01	`tenants/[id]/ad/users/[sam]` POST	Add `hasMinRole(role, "EDITOR")` check
M-02	`search/route.ts:11`	Add `q.length > 100` guard

Phase C — This Week: Security infrastructure

Finding	File	Action
H-01	`next.config.ts`	Add `headers()` with X-Frame-Options, CSP, etc.
H-02	`src/middleware.ts`	Integrate @upstash/ratelimit (distributed, serverless-safe)
H-04	`src/middleware.ts`	Validate JWT via auth() instead of cookie name check
H-05	Server actions	Add Zod schemas (SOC-2 Phase 4)
M-06	`src/lib/auth.ts`	Complete SOC-2 Phase 1: Vaultwarden integration + rotate secrets